

КУРС

Базы данных

PostgreSQL

Проектирование базы данных и ER- модель

Проектирование начинается не с CREATE TABLE

Слишком рано

CREATE TABLE ...

varchar(100)?

NULL или NOT NULL?

а что мы вообще храним?

Сначала смысл

что существует?

какие свойства?

как связано?

что обязательно?

Требования → модель → таблицы → SQL

Шаг 1. Выписываем факты из задачи

«Клиент делает заказы. Заказ содержит товары. Товар относится к категории. Для товара в заказе сохраняются количество и цена покупки»

Существительные

клиент заказ товар категория

Глаголы

делает содержит относится

Характеристики

количество цена покупки

Сначала выписываем факты. Типы данных - потом

Шаг 2. Находим сущности

customer

живёт независимо
есть свой id
делает заказы

product

есть в каталоге
есть свой id
имеет цену

orders

самостоятельный факт
есть свой id
имеет статус

category - тоже сущность: ее используют многие товары

Шаг 3. Выбираем атрибуты

сущность

атрибуты

customer

customer_id full_name email

product

product_id name price stock category_id

orders

order_id customer_id created_dt status

**Один атрибут =
один понятный
факт**

Плохо:
customer_info =
'Анна, +7999, Москва'

Детализация зависит от задач системы

Атрибут или отдельная сущность?

Атрибут

phone

одно значение

нет своих свойств

нужен только как контакт

Отдельная сущность

customer_phone

телефонов много

type · verified · created_dt

есть собственные правила

Сигнал: множество значений + свои свойства + свои правила

Шаг 4. Выбираем ключи

объект	ключ	почему
customer	customer_id	email может измениться
product	product_id	name не уникален
orders	order_id	нужна стабильная ссылка
category	category_id	name можно переименовать

order_item

вариант 1

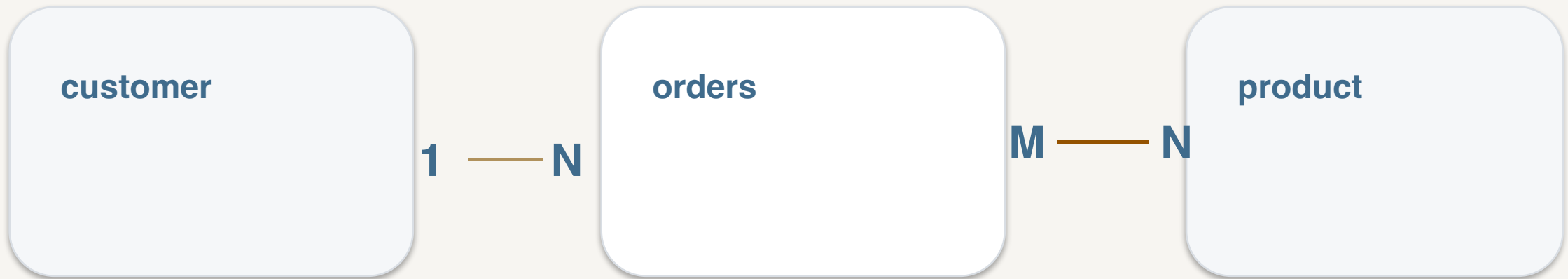
(order_id, product_id)

вариант 2

order_item_id

Ключ фиксирует правило уникальности

Шаг 5. Рисуем связи и кардинальность



Заказ ↔ товар = M:N

Кардинальность отвечает на вопрос: «сколько объектов с каждой стороны?»

Обязательность связи: может ли быть NULL?

Обязательная связь

`orders.customer_id`

NOT NULL

каждый заказ имеет клиента

гостевых заказов нет

Необязательная связь

`product.category_id`

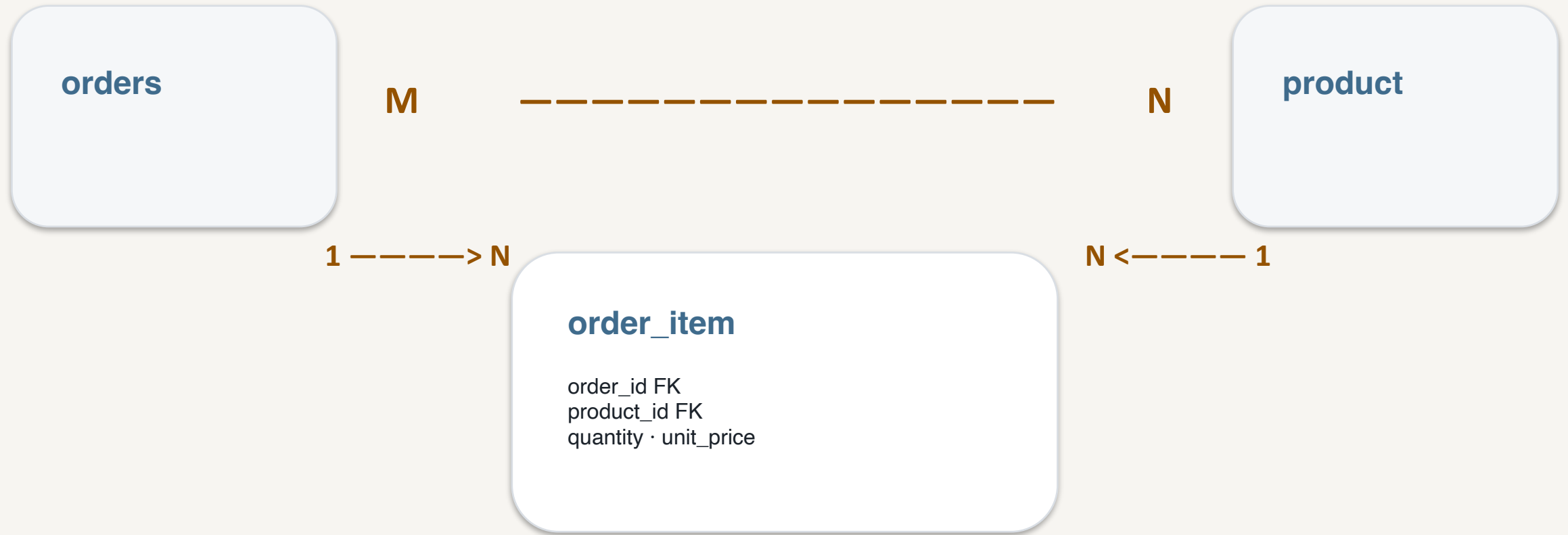
может быть NULL

товар-черновик без категории

назначим позже

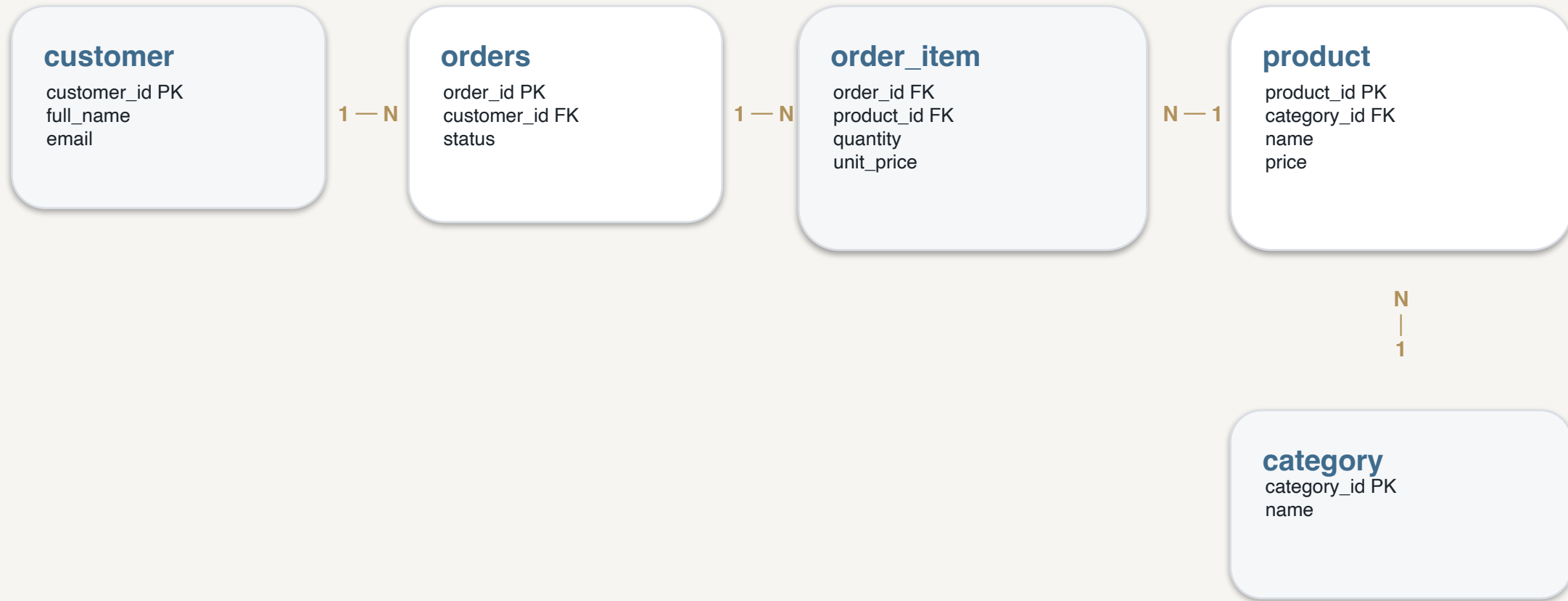
NULL допустим тогда, когда отсутствие значения имеет бизнес-смысл

M:N всегда требует решения



M:N → две связи 1:N + место для данных самой связи

ER-модель магазина



Читайте схему маршрутами: orders → customer orders → order_item → product

Логическая и физическая модель

Концептуальная

клиент

заказ

товар

СВЯЗИ ПО СМЫСЛУ

Логическая

атрибуты

ключи

1:N · M:N

нормализация

Физическая

bigint

numeric(10,2)

NOT NULL · FK

PostgreSQL

Сначала проверяем смысл и только потом выбираем детали реализации

Превращаем ER-модель в таблицы

```
CREATE TABLE customer (  
  customer_id bigint PRIMARY KEY,  
  full_name text NOT NULL,  
  email text NOT NULL UNIQUE  
);  
  
CREATE TABLE orders (  
  order_id bigint PRIMARY KEY,  
  customer_id bigint NOT NULL  
    REFERENCES customer(customer_id),  
  status text NOT NULL  
);
```

Как модель превращается в SQL

сущность → **таблица**

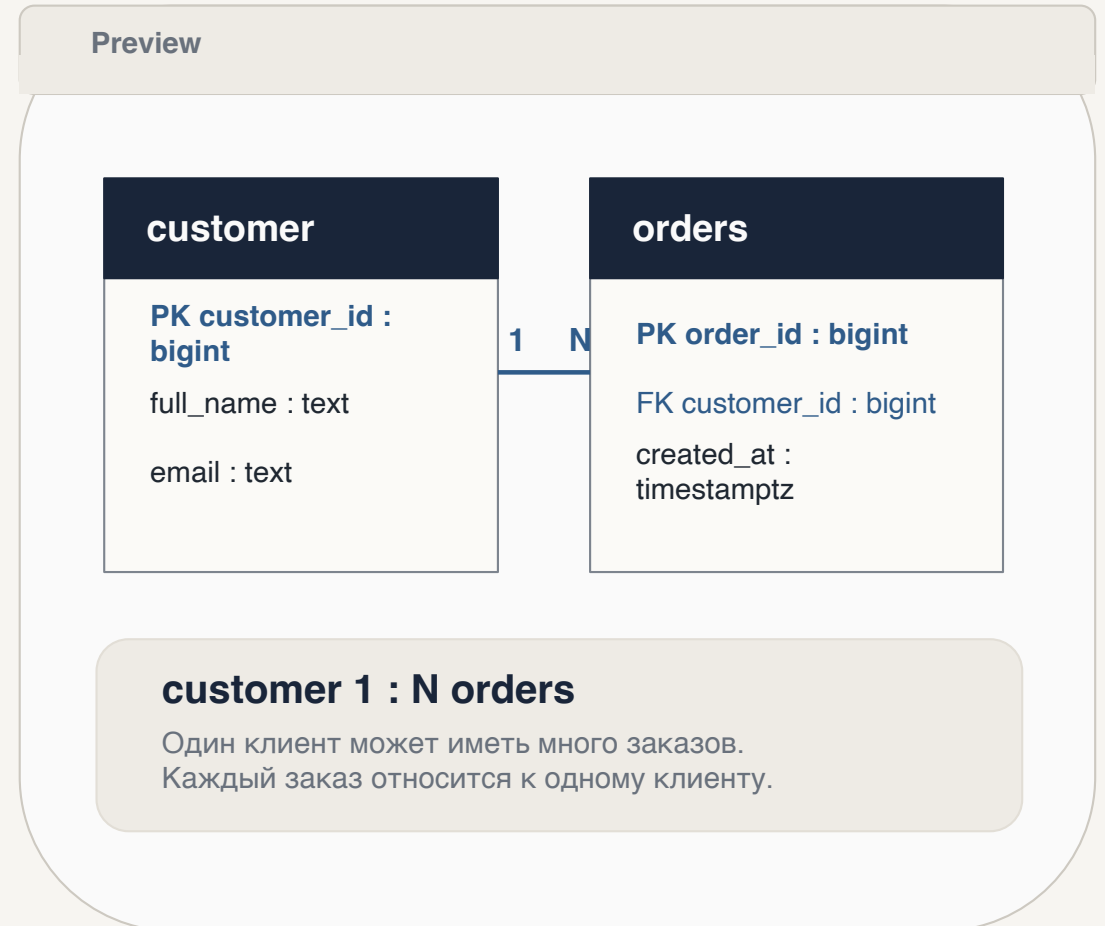
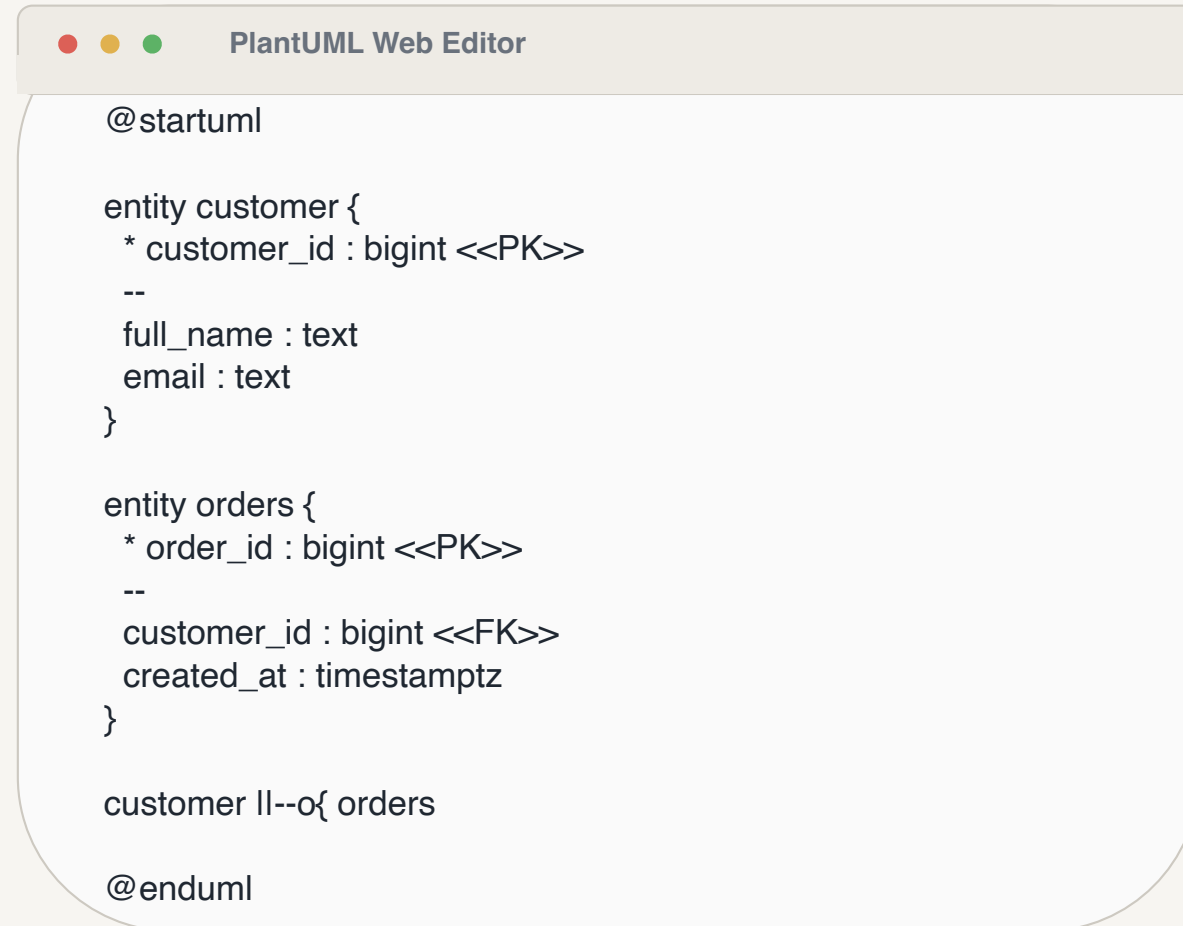
идентификатор → PRIMARY KEY

1:N → FK на стороне N

обязательность → NOT NULL

уникальность → UNIQUE

PlantUML: схема как текст



Проверяем модель на сценариях

сценарий

Анна делает 2 заказа

В заказе ноутбук + мышь

Цена товара изменилась

Категорию переименовали

модель отвечает

2 orders → 1 customer

2 строки order_item

старый unit_price сохранён

1 UPDATE в category

Тест модели

INSERT

можно добавить?

UPDATE

где меняем факт?

DELETE

что случайно
потеряем?

Типичные ошибки проектирования

Одна таблица

customer + order + product
всё повторяется

Список в поле

product_ids='1,2,3'
нет нормальных FK

Связи «на словах»

customer_id без FK
можно сослаться в пустоту

Еще ошибка: изменяемый текст как ключ и несколько «источников истины»

Добавляем новую сущность без переделки всего

Новая задача

Хранить попытки оплаты заказа

Не добавляем 20 полей в **orders**



payment

payment_id PK
order_id FK
amount
status
created_at

orders 1 — N
payment

Хорошая модель расширяется новой сущностью, а не бесконечным расширением существующих

КУРС

Базы данных

PostgreSQL

От ER-модели к работающей базе

ER-диаграмма становится настоящей БД

ER-модель

что существует и как связано

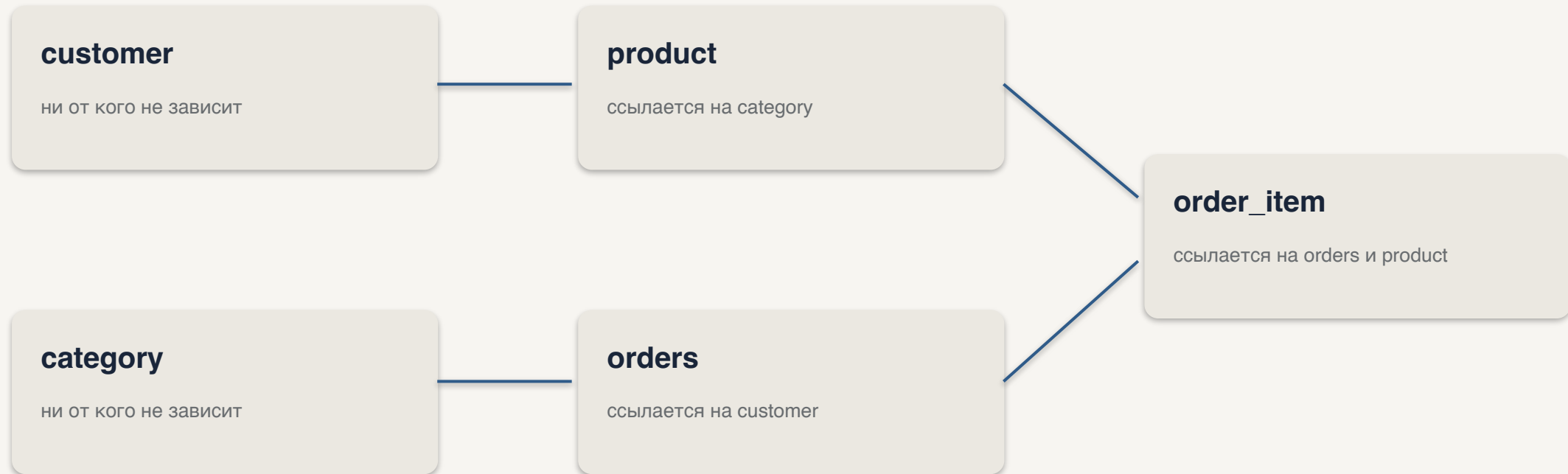
DDL

создаем таблицы и правила

Тест

правильные и неправильные данные

В каком порядке создавать таблицы



Сначала «родители», потом таблицы с FOREIGN KEY на них

Тип данных - это правило, а не оформление

bigint

идентификаторы

product_id = 101

integer

целое количество

stock = 15

numeric(10,2)

точные деньги

price = 79990.50

text

строка

name = Ноутбук

timestampz

момент времени

created_dt

boolean

да / нет

active = true

price numeric(10,2) stock integer created_dt timestampz

Начинаем с независимых таблиц

```
shop=# CREATE TABLE category (  
shop(#  category_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
shop(#  name text NOT NULL UNIQUE  
shop(# );  
CREATE TABLE  
  
shop=# CREATE TABLE customer (  
shop(#  customer_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
shop(#  full_name text NOT NULL,  
shop(#  email text NOT NULL UNIQUE  
shop(# );  
CREATE TABLE
```

Что уже защищено

PK — строка имеет id

NOT NULL — обязательное значение

UNIQUE — имя категории и email
не повторяются

Ограничения: база сама говорит «нельзя»

NOT NULL

значение обязательно

name

UNIQUE

не должно повторяться

email

CHECK

значение проходит условие

price >= 0

FK

ссылка существует

category_id

```
price numeric(10,2) NOT NULL CHECK (price >= 0)
stock integer NOT NULL DEFAULT 0 CHECK (stock >= 0)
```

Ограничение работает для сайта, импорта, скрипта и ручного SQL одинаково

Таблица product: собираем правила

```
shop=# CREATE TABLE product (  
shop#  product_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
shop#  category_id bigint NOT NULL REFERENCES category(category_id),  
shop#  name text NOT NULL,  
shop#  price numeric(10,2) NOT NULL CHECK (price >= 0),  
shop#  stock integer NOT NULL DEFAULT 0 CHECK (stock >= 0),  
shop#  active boolean NOT NULL DEFAULT true,  
shop#  CONSTRAINT uq_product_name_category UNIQUE(category_id,name)  
shop# );  
CREATE TABLE
```

Составное UNIQUE (category_id, name)

В одной категории нельзя два товара с одинаковым названием. Но одинаковое название в разных категориях допустимо.

Таблица orders: связь 1:N становится FOREIGN KEY

```
shop=# CREATE TABLE orders (  
shop(#  order_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
shop(#  customer_id bigint NOT NULL,  
shop(#  status text NOT NULL DEFAULT 'NEW',  
shop(#  created_at timestamptz NOT NULL DEFAULT now(),  
shop(#  CONSTRAINT fk_orders_customer  
shop(#    FOREIGN KEY(customer_id)  
shop(#      REFERENCES customer(customer_id)  
shop(#    ON DELETE RESTRICT,  
shop(#  CONSTRAINT chk_orders_status  
shop(#    CHECK (status IN ('NEW','PAID','DONE','CANCELLED'))  
shop(# );  
CREATE TABLE
```

Перевод

customer_id NOT NULL

заказ обязан иметь клиента

ON DELETE RESTRICT

клиента с заказами удалить нельзя

CHECK status

только известные статусы

ON DELETE: что делать со связанными строками

RESTRICT

не разрешать удаление

customer → orders

CASCADE

удалить зависимые строки

orders → order_item

SET NULL

оставить строку, убрать ссылку

необязательный manager_id

Выбор делаем по смыслу

Не существует «лучшего ON DELETE». Спросите: имеет ли дочерняя строка смысл без родительской?

Позиция без заказа — нет → CASCADE. Заказ без клиента в нашей модели — нет, но сам заказ терять нельзя → RESTRICT

order_item: два FOREIGN KEY и составное правило

```
shop=# CREATE TABLE order_item (  
shop(#  order_id bigint NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,  
shop(#  product_id bigint NOT NULL REFERENCES product(product_id) ON DELETE RESTRICT,  
shop(#  quantity integer NOT NULL CHECK (quantity > 0),  
shop(#  unit_price numeric(10,2) NOT NULL CHECK (unit_price >= 0),  
shop(#  CONSTRAINT pk_order_item PRIMARY KEY(order_id, product_id)  
shop(# );  
CREATE TABLE
```

Составной PK

(order_id, product_id)

Один товар — одна строка внутри заказа.

3 мышки:

quantity = 3

а не три одинаковые строки.

Заполняем базу правильными тестовыми данными

```
shop=# INSERT INTO category(name) VALUES ('Ноутбуки'),('Аксессуары');
INSERT 0 2
shop=# INSERT INTO customer(full_name,email)
shop=# VALUES ('Анна Иванова','anna@mail.ru');
INSERT 0 1
shop=# INSERT INTO product(category_id,name,price,stock) VALUES
shop=# (1,'Ноутбук',79990.50,10),(2,'Мышь',1490.00,50);
INSERT 0 2
shop=# INSERT INTO orders(customer_id,status) VALUES (1,'NEW') RETURNING order_id;
 order_id
-----
      1
(1 row)
```

data.sql

Данные должны проверять связи, а не просто заполнять таблицы.

Сначала родители, потом строки с FK.

Проверка CHECK: отрицательная цена не проходит

```
shop=# INSERT INTO product(category_id,name,price,stock)
shop=# VALUES (1,'Сломанный товар',-100.00,1);
ERROR: new row for relation "product" violates check constraint
"product_price_check"
```

Что доказали?

Тип numeric умеет хранить -100

Именно CHECK запрещает отрицательную цену по правилам магазина

Тип отвечает «что это за значение», CHECK — «разрешено ли такое значение в нашей предметной области»

Проверка UNIQUE: дубликат email не проходит

```
shop=# INSERT INTO customer(full_name,email)
shop=# VALUES ('Другая Анна','anna@mail.ru');
ERROR: duplicate key value violates unique constraint
        "customer_email_key"
DETAIL: Key (email)=(anna@mail.ru) already exists.
```

UNIQUE

не означает PRIMARY KEY

Но база гарантирует, что второй такой email не появится

Проверка FOREIGN KEY: ссылка в пустоту не проходит

```
shop=# INSERT INTO orders(customer_id,status)
shop=# VALUES (999,'NEW');
ERROR: insert or update on table "orders" violates foreign key constraint
      "fk_orders_customer"
DETAIL: Key (customer_id)=(999) is not present in table "customer".
```

FK проверяет

существует ли родительская строка

**Заказ не может ссылаться на
несуществующего клиента**

Ограничения проверяют и UPDATE

```
shop=# UPDATE product
shop=# SET stock = -5
shop=# WHERE product_id = 1;
ERROR: new row for relation "product" violates check constraint
"product_stock_check"

shop=# UPDATE orders SET status='SENT' WHERE order_id=1;
ERROR: violates check constraint "chk_orders_status"
```

Важно

Ограничение — не фильтр INSERT

Оно защищает состояние
таблицы как при INSERT,
так и при UPDATE

Три файла — три разные задачи

schema.sql

структура и правила

CREATE TABLE
PK · FK · CHECK

data.sql

корректные данные

INSERT
связанные строки

constraints_test.sql

намеренные ошибки

плохие INSERT
плохие UPDATE

Файл должен запускаться последовательно и воспроизводимо: **schema → data → tests**

КУРС

Базы данных

PostgreSQL

PostgreSQL и изменение структуры БД

База уже работает. А требования изменились.

Было

product: name · price · stock

orders: customer_id · status

в таблицах уже есть данные

Стало нужно

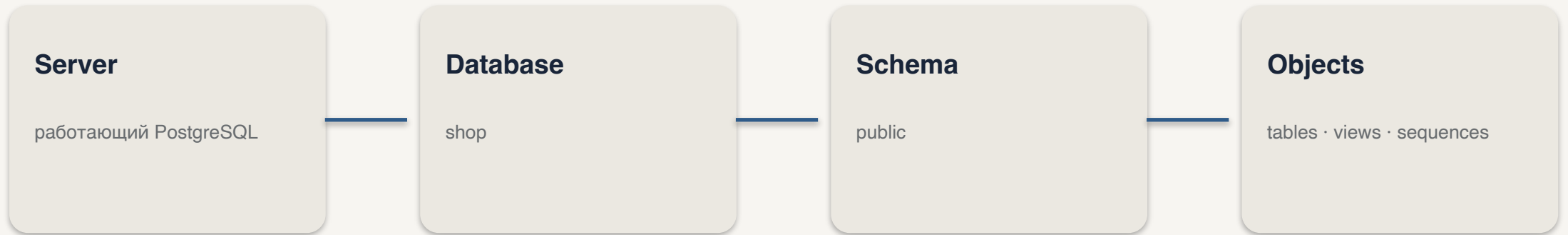
артикул sku у товара

новый статус RETURNED

доставка shipment

Нельзя: DROP DATABASE → создать заново, нужно изменить существующую структуру

Где живут наши объекты PostgreSQL



`shop.public.product` `shop.public.orders` `shop.public.order_item`

schema — пространство имен внутри базы, а не отдельная база данных

Сначала смотрим, что уже существует

```
shop=# \dt
```

List of relations

Schema	Name	Type
--------	------	------

public	category	table
--------	----------	-------

public	customer	table
--------	----------	-------

public	order_item	table
--------	------------	-------

public	orders	table
--------	--------	-------

public	product	table
--------	---------	-------

```
shop=# \d product
```

Column	Type	Nullable	Default
--------	------	----------	---------

product_id	bigint	not null	identity
------------	--------	----------	----------

name	text	not null	
------	------	----------	--

price	numeric(10,2)	not null	
-------	---------------	----------	--

Перед миграцией

проверяем текущую структуру

понимаем существующие данные

не предполагаем, что база пустая

ALTER TABLE: добавить новый столбец

```
shop=# ALTER TABLE product
shop=# ADD COLUMN sku text;
ALTER TABLE
```

```
shop=# SELECT product_id,name,sku FROM product;
product_id | name   | sku
-----+-----+-----
1          | Ноутбук | NULL
2          | Мышь   | NULL
(2 rows)
```

Почему сначала NULL?

Старые строки уже существуют.

Если сразу потребовать неизвестный sku, существующим данным нечего подставить.

Безопасная схема: add → fill → constrain

1. ADD

добавить nullable столбец

2. FILL

заполнить старые строки

3. CONSTRAIN

добавить NOT NULL / UNIQUE

```
ALTER TABLE product ADD COLUMN sku text;  
UPDATE product SET sku = 'SKU-' || product_id WHERE sku IS NULL;  
ALTER TABLE product ALTER COLUMN sku SET NOT NULL;
```

Сначала заботимся о существующих строках и только потом ужесточаем правило

UPDATE заполняет существующие строки

```
shop=# UPDATE product
shop=# SET sku = 'SKU-' || product_id
shop=# WHERE sku IS NULL;
UPDATE 2
```

```
shop=# SELECT product_id,name,sku FROM product ORDER BY product_id;
 product_id | name   | sku
-----+-----+-----
1           | Ноутбук | SKU-1
2           | Мышь   | SKU-2
(2 rows)
```

Миграция

может менять не только структуру, но и данные, если это нужно для перехода к новой структуре.

Главное — результат должен быть согласован.

Теперь можно включить NOT NULL

```
shop=# ALTER TABLE product
shop=# ALTER COLUMN sku SET NOT NULL;
ALTER TABLE

shop=# INSERT INTO product(category_id,name,price)
shop=# VALUES (1,'Планшет',39990.00);
ERROR: null value in column "sku" violates not-null constraint
```

Порядок важен

Если старые строки заполнены -
правило можно ужесточить.

**Новые строки уже обязаны
передавать sku.**

Добавляем UNIQUE отдельным ограничением

```
shop=# ALTER TABLE product
shop=# ADD CONSTRAINT uq_product_sku UNIQUE (sku);
ALTER TABLE

shop=# UPDATE product SET sku='SKU-1' WHERE product_id=2;
ERROR: duplicate key value violates unique constraint "uq_product_sku"
```

ADD CONSTRAINT

добавляет правило к уже существующей таблице

PostgreSQL проверяет и существующие данные

Удаляем столбец

Бизнес-ситуация

Раньше описание товара хранили в product.description.
Теперь тексты ведёт отдельная CMS. Поле в PostgreSQL больше не используется.

Что хотим получить

Убрать description из product, не затрагивая остальные данные товара.

Перед удалением

Проверяем код приложения, VIEW, функции и запросы: никто больше не должен читать этот столбец.

```
psql · shop

shop=# SELECT product_id, name, description
shop=# FROM product
shop=# ORDER BY product_id;

product_id | name          | description
-----+-----+-----
1 | Ноутбук      | 16 ГБ, 512 ГБ SSD
2 | Мышь         | Беспроводная
3 | Монитор      | 27 дюймов
(3 rows)

shop=# ALTER TABLE product
shop=# DROP COLUMN description;
ALTER TABLE
```

```
psql · shop

shop=# \d product

          Table "public.product"
   Column |      Type      | Nullable
-----+-----+-----
product_id | bigint         | not null
category_id | bigint         | not null
name       | text           | not null
price      | numeric(10,2)  | not null
stock      | integer        | not null
active     | boolean        | not null

Indexes:
  "product_pkey" PRIMARY KEY, btree (product_id)

-- description больше нет
```

ALTER TABLE product DROP COLUMN description; → **удаляется и сам столбец, и значения, которые в нём хранились.**

CHECK изменяем через DROP + ADD

```
-- было: NEW, PAID, DONE, CANCELLED
shop=# ALTER TABLE orders
shop=# DROP CONSTRAINT chk_orders_status;
ALTER TABLE

shop=# ALTER TABLE orders
shop=# ADD CONSTRAINT chk_orders_status
shop=# CHECK (status IN ('NEW','PAID','DONE','CANCELLED','RETURNED'));
ALTER TABLE
```

Почему так?

У CHECK есть определение.
Чтобы изменить условие,
старое ограничение удаляем и
создаем новое.

Данные при этом не пересоздаем

Меняем правило ON DELETE

```
shop=# ALTER TABLE orders  
shop=# DROP CONSTRAINT fk_orders_customer;  
ALTER TABLE
```

```
shop=# ALTER TABLE orders  
shop=# ADD CONSTRAINT fk_orders_customer  
shop=# FOREIGN KEY(customer_id)  
shop=# REFERENCES customer(customer_id)  
shop=# ON DELETE RESTRICT;  
ALTER TABLE
```

FK — тоже constraint

Его действие можно изменить через
DROP CONSTRAINT +
ADD CONSTRAINT

**Сначала понимаем последствия
для существующих данных**

Новая сущность: добавляем shipment

```
shop=# CREATE TABLE shipment (  
shop# shipment_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
shop# order_id bigint NOT NULL UNIQUE,  
shop# shipped_dt timestamptz,  
shop# CONSTRAINT fk_shipment_order  
shop# FOREIGN KEY(order_id) REFERENCES orders(order_id) ON DELETE CASCADE  
shop# );  
CREATE TABLE
```

UNIQUE(order_id)

делает связь orders ↔ shipment

фактически 1:1

**Один заказ - не более одной
строки shipment**

Переносим столбец в другую таблицу — вместе со значениями

Было

orders хранит и заказ, и адрес доставки:
order_id, customer_id,
delivery_address

Почему меняем

У доставки появляется собственная логика: статус, время отправки, трек-номер. Значит, shipment становится отдельной сущностью.

Станет

orders — заказ
shipment — доставка + address

psql · shop

```
BEGIN;  
  
ALTER TABLE shipment  
ADD COLUMN address text NOT NULL;  
  
INSERT INTO shipment (order_id, address)  
SELECT order_id, delivery_address  
FROM orders;  
  
ALTER TABLE orders DROP COLUMN delivery_address;  
  
COMMIT;
```

psql · shop

```
shop=# SELECT *  
shop=# FROM shipment;  
  
shipment_id | order_id | address  
-----+-----+-----  
1 | 101 | Москва, Ленина 8  
2 | 102 | Москва, Мира 15  
3 | 103 | Химки, Молодёжная 4  
(3 rows)  
  
shop=# \d orders  
...  
customer_id | bigint  
status      | text  
created_dt  | timestamptz  
  
-- delivery_address  
-- больше нет
```

Критический порядок: создать новое место → скопировать данные → проверить → только потом удалить старый столбец.

RENAME

```
shop=# ALTER TABLE customer  
shop-# RENAME COLUMN full_name TO name;  
ALTER TABLE  
  
shop=# ALTER TABLE shipment  
shop-# RENAME COLUMN address TO delivery_address;  
ALTER TABLE
```

RENAME

данные остаются на месте

**но приложения и SQL, которые
используют старое имя,
нужно обновить**

DROP COLUMN / DROP CONSTRAINT

```
ALTER TABLE product DROP COLUMN old_code;  
ALTER TABLE product DROP CONSTRAINT uq_product_sku;
```

Перед DROP спросить

есть ли данные в столбце?

кто читает этот столбец?

не зависит ли от него view / код / отчёт?

Правило

ADD обычно обратим проще

DROP может уничтожить информацию

DDL в PostgreSQL можно объединять транзакцией

```
shop=# BEGIN;  
BEGIN  
shop=# ALTER TABLE product ADD COLUMN weight_kg numeric(8,3);  
ALTER TABLE  
shop=# ROLLBACK;  
ROLLBACK
```

Полезно

Многие DDL-операции PostgreSQL транзакционны

Можно проверить набор изменений и откатить его целиком

Но влияние блокировок и тяжелых ALTER в больших БД — отдельная тема

migration.sql: изменение как последовательный сценарий

```
BEGIN;  
  
ALTER TABLE product ADD COLUMN sku text;  
UPDATE product SET sku='SKU-' || product_id WHERE sku IS NULL;  
ALTER TABLE product ALTER COLUMN sku SET NOT NULL;  
ALTER TABLE product ADD CONSTRAINT uq_product_sku UNIQUE(sku);  
  
ALTER TABLE orders DROP CONSTRAINT chk_orders_status;  
ALTER TABLE orders ADD CONSTRAINT chk_orders_status  
  CHECK (status IN ('NEW','PAID','DONE','CANCELLED','RETURNED'));  
  
COMMIT;
```

Хорошая миграция

меняет существующую БД

сохраняет данные

выполняется в понятном порядке

**после неё схема соответствует
новым требованиям**

DDL

DDL — Data Definition Language

команды, которые создают, изменяют и удаляют объекты базы данных

CREATE

создать объект

```
CREATE TABLE shipment (...)
```

ALTER

изменить объект

```
ALTER TABLE product  
ADD COLUMN sku text;
```

DROP

удалить объект

```
DROP TABLE old_table;
```

TRUNCATE

очистить таблицу

```
TRUNCATE TABLE temp_import;
```

DDL меняет структуру. INSERT / UPDATE / DELETE / SELECT работают с данными внутри этой структуры.

DDL в нашем магазине

psql · shop

```
shop=# CREATE TABLE promo_code (  
shop(#  promo_code_id bigint  
shop(#      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
shop(#  code text NOT NULL UNIQUE,  
shop(#  discount_percent integer NOT NULL  
shop(#      CHECK (discount_percent BETWEEN 1 AND 90)  
shop(# );  
CREATE TABLE  
  
shop=# ALTER TABLE promo_code  
shop-# ADD COLUMN active boolean NOT NULL DEFAULT true;  
ALTER TABLE  
  
shop=# \d promo_code
```

psql · shop

```
Table "public.promo_code"  
Column | Type | Nullable | Default  
-----+-----+-----+-----  
promo_code_id | bigint | not null | generated always  
code | text | not null |  
discount_percent | integer | not null |  
active | boolean | not null | true  
  
Indexes:  
"promo_code_pkey" PRIMARY KEY, btree (promo_code_id)  
"promo_code_code_key" UNIQUE, btree (code)  
  
Check constraints:  
discount_percent >= 1  
AND discount_percent <= 90
```

CREATE создал таблицу. ALTER изменил ее. \d показывает, какой стала структура после выполнения DDL

Автоинкрементные идентификаторы: SERIAL и IDENTITY

Оба варианта выдают следующий номер автоматически, но устроены и описываются по-разному.

SERIAL

Не настоящий тип данных, а удобная сокращённая запись.
PostgreSQL создаёт integer + sequence + DEFAULT nextval(...).

psql · shop

```
CREATE TABLE supplier (  
  supplier_id serial PRIMARY KEY,  
  name text NOT NULL  
);
```

```
INSERT INTO supplier (name)  
VALUES ('ТехноОпт'), ('ДисплейПро')  
RETURNING supplier_id, name;
```

```
supplier_id | name  
-----+-----  
1 | ТехноОпт  
2 | ДисплейПро
```

GENERATED AS IDENTITY

Стандартный SQL-механизм для автоматически генерируемого идентификатора.
Последовательность управляется как часть identity-столбца.

psql · shop

```
CREATE TABLE product (  
  product_id bigint  
    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name text NOT NULL  
);
```

```
INSERT INTO product (name)  
VALUES ('Ноутбук'), ('Мышь')  
RETURNING product_id, name;
```

```
product_id | name  
-----+-----  
1 | Ноутбук  
2 | Мышь
```

IDENTITY: ALWAYS или BY DEFAULT?

Разница — разрешаем ли мы вручную подставить значение идентификатора

GENERATED ALWAYS

Обычный INSERT не должен задавать id вручную.

База сама отвечает за идентификатор.

psql · shop

```
CREATE TABLE category (  
  category_id bigint  
    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name text NOT NULL  
);
```

```
INSERT INTO category (name)  
VALUES ('Ноутбуки')  
RETURNING category_id;
```

category_id

Почему это удобно

Приложение не думает, какой номер следующий.

Не нужно вычислять $\text{MAX}(\text{id}) + 1$ — это небезопасно при параллельных INSERT.

GENERATED BY DEFAULT

База генерирует id, если мы его не передали.

Но явное значение разрешено.

psql · shop

```
id bigint  
GENERATED BY DEFAULT AS IDENTITY
```

```
INSERT INTO demo (id, name)  
VALUES (100, 'Импорт');
```

```
-- id = 100 допустим
```

Когда BY DEFAULT полезен

Например, при переносе данных из старой системы, когда нужно сохранить прежние идентификаторы.

UUID: идентификатор без общего счётчика

UUID — не автоинкремент. Это отдельный 128-битный тип данных; значение генерируется как уникальный идентификатор.

Как выглядит

a3f8721c-8b42-4f61-9c31-90d17f3e28aa

Не 1, 2, 3... — номера не идут по порядку.

Зачем нужен

Разные серверы или приложения могут создавать идентификаторы независимо, без единого «следующего номера».

Цена

UUID занимает 16 байт; BIGINT — 8 байт. Случайный UUID хуже читается человеком.

psql · shop

```
CREATE TABLE api_order (  
  order_id uuid  
    DEFAULT gen_random_uuid()  
  PRIMARY KEY,  
  customer_id bigint NOT NULL  
    REFERENCES customer(customer_id)  
);
```

```
INSERT INTO api_order (customer_id)  
VALUES (1)  
RETURNING order_id;
```

order_id
a3f8721c-8b42-4f61-9c31-90d17f3e28aa

(1 row)

Главное различие

IDENTITY

1 → 2 → 3 → 4

центральная последовательность

UUID

генерируется независимо
практически уникален
не обязан быть
последовательным